

Docker Training

Day 1 · Session 1

Docker mental model & architecture

9:00 – 10:30 · 1.5 hours · Theory + Hands-on Lab

Session objectives

- Understand the Docker architecture: daemon, CLI, images, containers, registry
- Know the difference between containers and virtual machines
- Pull images from Docker Hub and Microsoft Container Registry
- Run, inspect, exec into, and remove containers
- Read image layers and understand the union filesystem model

Key concepts

Concept	What it means
Docker daemon	Background service (dockerd) that manages all containers and images. The CLI talks to it via a Unix socket.
Docker CLI	The docker command you type. Sends REST API calls to the daemon. It is not the engine itself.
Image	Read-only blueprint made of stacked layers. Like a class in OOP — you never run an image directly.
Container	A running instance of an image. Like an object instantiated from a class. Ephemeral by default.
Registry	Remote store for images. Docker Hub is the default. GHCR, ECR, and ACR are private alternatives.
Layer / OverlayFS	Each Dockerfile instruction adds a read-only layer. Layers are shared across images — huge disk space saving.
Namespace	Linux kernel feature that isolates the container's view of processes, network, filesystem, and users.
cgroup	Linux kernel feature that limits CPU and memory resources available to a container.

Hands-on lab steps

1

Install Docker & verify the engine

Make sure everyone has a working Docker installation before touching any images

Windows / Mac — Docker Desktop

```
# Download from https://www.docker.com/products/docker-desktop/  
# After install, open a terminal and verify:
```

```
docker version
```

Expected output:

```
Client: Docker Engine - Community  
Version: 26.x.x  
Server: Docker Engine - Community  
Engine:  
Version: 26.x.x  
API version: 1.45 (minimum version 1.24)
```

Ubuntu / Debian — Docker Engine

```
sudo apt-get update && sudo apt-get install ca-certificates curl  
sudo install -m 0755 -d /etc/apt/keyrings  
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \  
-o /etc/apt/keyrings/docker.asc  
sudo chmod a+r /etc/apt/keyrings/docker.asc  
  
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-compose-  
plugin  
  
# Allow running docker without sudo (requires logout/login)  
sudo usermod -aG docker $USER
```

Verify daemon info

```
docker info
```

What is happening under the hood

docker version shows both the client (CLI) and server (daemon) versions. Two versions must appear — client AND server. If only one shows, the daemon is not running. On Linux run: `sudo systemctl start docker`

Trainer tip

Ask everyone to run `docker version` and show you the output. Two blocks must appear — Client and Server.

2

Pull official images from Docker Hub

Introduce the registry concept and image naming scheme

Pull images for your stack

```
# Pull PostgreSQL 16 official image  
docker pull postgres:16  
  
# Pull .NET 8 ASP.NET runtime
```

```
docker pull mcr.microsoft.com/dotnet/aspnet:8.0

# Pull Node 20 Alpine (small, for Angular builds)
docker pull node:20-alpine

# List all locally available images
docker images
```

Expected output:

REPOSITORY	TAG	IMAGE ID	CREATED
SIZE			
postgres	16	a1b2c3d4e5f6	2 days ago
432MB			
mcr.microsoft.com/dotnet/aspnet	8.0	b2c3d4e5f6a7	5 days ago
217MB			
node	20-alpine	c3d4e5f6a7b8	1 week ago
135MB			

What is happening under the hood

Image name format: registry/repository:tag. When no registry is specified, Docker Hub is assumed. mcr.microsoft.com is Microsoft's Container Registry — official .NET images always come from there. The tag pins the version. Never rely on 'latest' in production — it silently changes.

Trainer tip

Point out that all three images have different IDs even though they're all Linux. Ask why — segue into the layer model discussion.

3

Run PostgreSQL as a container

First real container run — show port mapping, environment variables, detached mode

Run postgres in detached mode

```
docker run -d \
  --name pg \
  -e POSTGRES_PASSWORD=Training123 \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_DB=traindb \
  -p 5432:5432 \
  postgres:16
```

Expected output:

```
a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6 (container ID)
```

Confirm it is running

```
docker ps
```

Expected output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
NAMES					
a1b2c3d4e5f6	postgres:16	"docker-en..."	5 seconds ago	Up 4 seconds	
0.0.0.0:5432->5432/tcp	pg				

What is happening under the hood

-d runs in detached (background) mode. --name pg gives the container a memorable name. -e sets environment variables. -p 5432:5432 maps host port to container port (format: host:container). Connect from DBeaver: host=localhost, port=5432, user=postgres, password=Training123.

Trainer tip

Live demo: connect DBeaver from the host to the containerized PostgreSQL. This is the moment where the value of containers clicks for the group.

4

Inspect image layers with docker history & docker inspect

Make the 'images are layered' concept concrete and visible

See the layers that make up postgres:16

```
docker history postgres:16
```

Inspect the running container (full JSON)

```
docker inspect pg
```

Filter inspect output with --format

```
# Get just the container IP address
docker inspect pg --format '{{.NetworkSettings.IPAddress}}'

# Get just the running state
docker inspect pg --format '{{.State.Status}}'
```

What is happening under the hood

docker history shows every layer and the command that created it, read bottom-to-top. The base debian:bookworm-slim layer is shared with every other Debian-based image on your machine — stored once on disk. docker inspect returns the full configuration JSON. This is the source of truth for debugging: IPs, mounts, env vars, port bindings.

Warning — tell the group

POSTGRES_PASSWORD is visible in plain text in docker inspect output. Note this now — proper secrets management is covered in Day 2 Session 4.

5

Exec into a running container

Show that a container is a real isolated Linux process, not a VM

Open a shell inside the postgres container

```
docker exec -it pg bash
```

```
# Now INSIDE the container — run these:
cat /etc/os-release      # Which Linux distro?
ps aux                  # What processes are running?
env | grep POSTGRES      # See environment variables
ls /var/lib/postgresql/  # Postgres data directory
exit                    # Return to host
```

Connect to PostgreSQL directly inside the container

```
docker exec -it pg psql -U postgres traindb

# Inside psql — run these SQL commands:
\l                                -- list all databases
\dt                               -- list tables (empty for now)
CREATE TABLE test (id SERIAL PRIMARY KEY, name TEXT);
INSERT INTO test (name) VALUES ('Docker training');
SELECT * FROM test;
\q                                -- quit psql
```

What is happening under the hood

`docker exec -it` runs a new process inside an already-running container. `-i` keeps stdin open, `-t` allocates a pseudo-TTY. Notice how few processes appear in `ps aux` inside the container vs hundreds on the host — this is Linux namespaces isolating the process tree.

Trainer tip

Ask: 'What do you see in `ps aux` inside the container?' — only postgres worker processes. Compare to the host. This makes namespaces tangible.

6

Clean up — stop, remove, prune

Teach the container lifecycle and how to keep the environment clean

Container lifecycle commands

```
docker stop pg           # Graceful stop (SIGTERM, then SIGKILL after 10s)
docker start pg          # Restart it — data is still there!
docker rm -f pg          # Force stop and remove
docker rmi postgres:16   # Remove the image

# List ALL containers including stopped ones
docker ps -a

# Run a one-shot container that removes itself on exit
docker run --rm postgres:16 postgres --version
```

Nuclear cleanup — remove everything unused

```
docker system prune -a --volumes
```

Expected output:

```
Total reclaimed space: 632MB
```

What is happening under the hood

docker stop sends SIGTERM, then SIGKILL after 10 seconds. docker rm removes the container but not the image. Any data written to the container layer (not a volume) is lost on rm. docker system prune removes all stopped containers, dangling images, unused networks, and optionally volumes.

Warning — tell the group

docker rm -f and docker system prune are destructive and immediate. There is no undo. Always run docker ps -a first to see what you are about to lose.

Quick reference — essential commands

Command	What it does
<code>docker version</code>	Check CLI and daemon versions
<code>docker info</code>	Full daemon status and configuration
<code>docker pull <image>:<tag></code>	Download an image from a registry
<code>docker images</code>	List locally available images
<code>docker run -d --name X -p H:C image</code>	Run a container in background with port mapping
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers (including stopped)
<code>docker exec -it <name> bash</code>	Open an interactive shell inside a container
<code>docker stop <name></code>	Gracefully stop a running container
<code>docker rm -f <name></code>	Force stop and remove a container
<code>docker rmi <image></code>	Remove a local image
<code>docker history <image></code>	Show image layers
<code>docker inspect <name></code>	Full container/image details as JSON
<code>docker system prune -a --volumes</code>	Remove all unused Docker objects

Coming up next — Session 2

Writing Dockerfiles for .NET Web API and Angular

- Multi-stage builds to minimize image size
- Layer caching strategy — why order matters
- .dockerignore — what to exclude from the build context
- Alpine vs Debian base images trade-offs